Askapeer — Technical Architecture Specification

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: Cross-cutting technical architecture underpinning all MVP epics. This document does **not** replace the per-epic technical specs (one per EPIC-A through EPIC-I) — it establishes the shared foundation those specs will build on: technology stack, hosting, data model, identity/pseudonymity enforcement, authentication, and non-functional requirements.

Source of truth for product requirements: `docs/askapeer-prd-v0.1.md`.

Amendment, 14 July 2026 (post-approval): `identity.members.verification_status` (Section 4.1) gains an `expelled` value, resolving a gap identified while drafting the EPIC-B and EPIC-F specs — see `docs/2026-07-14-technical-specs-open-questions.md`, Section 2, for the full history.

Contents

---

1. Assumptions and scope notes

The PRD (`docs/askapeer-prd-v0.1.md`, Section 15) lists eight open stakeholder decisions (FD-1 through FD-8) that are not yet formally closed. Per direction from the technical lead, this architecture is designed **against the PRD's stated recommendations** for each open item, so design work is not blocked on sign-off. Each assumption below should be treated as revisable, not settled, until the relevant stakeholders close the FD item:

FD item	Assumption made here
FD-1 (professional scope)	Physio-first MVP, per the PRD's Option B recommendation

FD item	Assumption made here
FD-3 (platform sequencing)	Web-first for MVP; architecture is API-first specifically so native iOS/ Android (Phase 2) requires no backend rework
FD-4 (forum taxonomy)	Hybrid model (Option D): fixed top-level categories plus free tagging
FD-5 (1:1 messaging)	Deferred to Phase 2, per the PRD's recommendation

FD-2 (subscription pricing/processor) and FD-7/FD-8 (branding, competitive research) have no material architectural impact and are not assumed here; FD-6 (university partnership) is an operational, not technical, decision.

One scope addition beyond the PRD: this document adds a ninth epic, **EPIC-I — Research/News Feed**, covering the personalised research-literature feed prototyped in `prototypes/research-feed/` and designed in `docs/AHP_Research_Feed_Design_Conversation.md`. This was not in the PRD's original MoSCoW list (Section 6.1) and should be reflected back into the PRD the next time it's revised with Paul Gouge and Andrew Renshaw.

---

2. Glossary

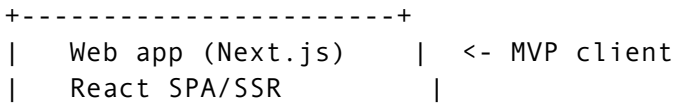
Acronyms are expanded at first use in the body text below; this table is a standing reference. Terms already defined in the PRD's own glossary (Appendix B) are not repeated in full here.

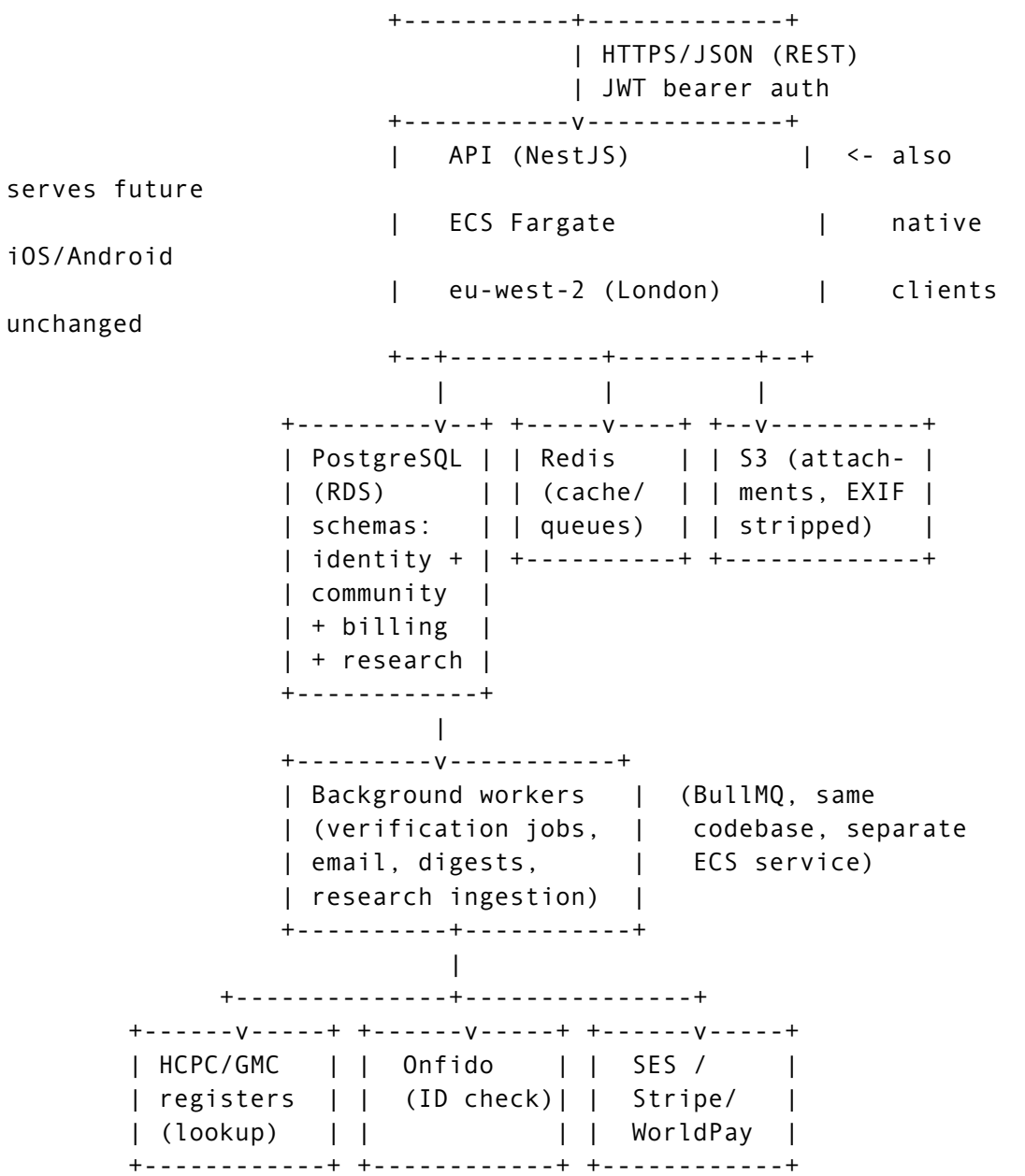
Term	Meaning
API	Application Programming Interface — a defined way for software components to communicate, here mainly over HTTPS using JSON
REST	Representational State Transfer — an API design style built around resources and standard HTTP verbs
JSON	JavaScript Object Notation — a lightweight text format for structured data
JWT	JSON Web Token — a signed, compact token format used here to represent an authenticated session
DTO	Data Transfer Object — the shape of data returned by an API endpoint, distinct from how it's stored in the database
TLS	Transport Layer Security — encrypts data in transit between client and server (the successor to SSL)
AWS	

Term	Meaning
	Amazon Web Services — the cloud hosting provider specified in the PRD
VPC	Virtual Private Cloud — an isolated private network within AWS
ALB	Application Load Balancer — distributes incoming web traffic across running application instances
NAT gateway	Network Address Translation gateway — lets resources in a private network reach the internet without being directly reachable from it
ECS	Elastic Container Service — AWS's service for running containerised applications
Fargate	The "serverless" compute mode for ECS — no servers to manage directly
RDS	Relational Database Service — AWS's managed database hosting
S3	Simple Storage Service — AWS's file/object storage
CDN	Content Delivery Network — caches and serves content from locations close to the user (here, AWS CloudFront)
WAF	Web Application Firewall — filters malicious web traffic before it reaches the application
CDK	Cloud Development Kit — a way of defining AWS infrastructure as code, here in TypeScript
CI/CD	Continuous Integration / Continuous Deployment — automated building, testing, and deploying of code changes
ECR	Elastic Container Registry — AWS's storage for container images
PII	Personally Identifiable Information — data that could identify a specific real person
PHI	Protected Health Information — identifiable patient data (see PRD glossary); never permitted on the platform
GDPR	General Data Protection Regulation — UK/EU data protection law
DPIA	Data Protection Impact Assessment — a formal review of privacy risk,

Term	Meaning
	recommended before handling data like Askapeer's at scale
DOI	Digital Object Identifier — a persistent unique identifier for an academic publication
PMID	PubMed Identifier — a unique identifier assigned by the PubMed database
EXIF	Exchangeable Image File Format — metadata embedded in photos (e.g. GPS location) that must be stripped before storage
SES	Simple Email Service — AWS's email-sending service
PCI	Payment Card Industry (Data Security Standard) — compliance requirements for handling card payments, offloaded here to the payment processor
SPA	Single-Page Application — a web app that runs mostly in the browser rather than reloading pages from the server
SSR	Server-Side Rendering — generating a web page's HTML on the server before sending it to the browser
SEO	Search Engine Optimisation — making pages discoverable by search engines
GIN index	Generalized Inverted Index — a PostgreSQL index type used here for full-text search
RPO	Recovery Point Objective — the maximum acceptable amount of data loss, measured in time, after an outage
RTO	Recovery Time Objective — the maximum acceptable time to restore service after an outage
APM	Application Performance Monitoring — tooling for tracking application errors and performance in production

3. System overview and components





Stack decision: Node.js/TypeScript throughout, using the NestJS framework for the Application Programming Interface (API), organised as a **modular monolith** (not microservices) — the right size for a small team, avoiding distributed-systems overhead while still giving each module (“epic”) a clean, testable boundary. TypeScript end-to-end means the web frontend and backend share types, and it’s the stack AI coding agents are most fluent in, which matters given the team is building with AI assistance.

Components: - **Web app** — Next.js (React), the sole MVP client, communicating with the API entirely over HTTPS. Built API-first (no server-only logic baked into the web app) so a future native mobile client is architecturally just another API consumer, not a rewrite. - **API** — a single NestJS service, organised into modules mapping 1:1 to the PRD epics (see Section 10). - **Background workers** — the same codebase, deployed as a second Elastic Container Service (ECS) service, driven by a Redis-backed job queue (BullMQ). Handles anything that shouldn’t block a request: professional-register lookups, identity-check polling, email sending, EXIF stripping, digest generation, research-feed ingestion. - **PostgreSQL** (Relational Database Service (RDS), Multi-AZ, `eu-west-2`/London) — a single instance with four schemas: `identity`, `community`, `billing`, `research`

(Section 4). - **Redis** (ElastiCache) — job queue plus hot-path caching (e.g. kudos leaderboards). - **S3** (Simple Storage Service) — attachments and verification documents in private buckets, accessed via presigned Uniform Resource Locators (URLs); EXIF metadata stripped by a worker before anything is persisted.

---

4. Data model and the identity/pseudonymity boundary

The core architectural constraint (PRD Section 9) is: **no path exists from a handle to a real identity except through one audited, narrow service.** This is enforced two ways — schema separation *and* PostgreSQL role-based access grants (not application-code discipline alone, which a bug or a careless new module could bypass).

4.1 `identity` schema — real personally identifiable information (PII)

Only three internal modules ever hold a database role with access here: `IdentityService` (verification, moderator-initiated lookups), `BillingService` (invoicing), and `NotificationService` (email delivery). No other module's database role has grants on this schema — enforced by PostgreSQL `GRANT`/`REVOKE`, verified by an automated test (Section 9).

```
identity.members
  id                    uuid PK          -- the real "member_id"
  legal_name            text
  email                 text unique
  professional_body     enum(hcpc, gmc, basrat, sst)
  registration_number   text
  registration_country  text
  verification_status   enum(pending, needs_more_info,
approved_verified, rejected, suspended, expelled)
  status_updated_at     timestamptz
  created_at            timestamptz

identity.verification_evidence
  id            uuid PK
  member_id     uuid FK -> identity.members
  evidence_type enum(register_lookup, onfido_check,
manual_document)
  source        text            -- e.g. "HCPC register API",
"Onfido"
  raw_result    jsonb
  outcome       enum(pass, fail, needs_review)
  created_at    timestamptz

identity.verification_decisions        -- immutable: INSERT-only
grant
  id            uuid PK
  member_id     uuid FK -> identity.members
  from_status   text
```

```
  to_status     text
  decided_by    text            -- 'system' or an admin's
member_id
  reason        text
  created_at    timestamptz

identity.case_attestations             -- see 4.3
  id                 uuid PK
  member_id          uuid FK -> identity.members
  post_id            uuid FK -> community.posts
  attestation_text   text
  checklist_snapshot jsonb
  attested_at        timestamptz
  ip_address         inet

identity.identity_access_log           -- immutable: INSERT-only
grant
  id                uuid PK
  accessed_by       uuid            -- moderator/admin's own
member_id
  target_member_id  uuid FK -> identity.members
  reason_code       enum(reported_violation, legal_request,
safety_escalation)
  reason_note       text
  action_taken      text
  created_at        timestamptz
```

4.2 `community` schema — everything peer-facing

Read/written by the `forum`, `kudos`, `case-discussions`, `moderation` modules. This database role has **zero** grants on the `identity` schema.

```
community.handles
  id           uuid PK          -- the public "handle_id"
  member_id    uuid             -- FK exists for referential
integrity only;
                                  -- readable solely via
IdentityService's role
  handle_name  text unique
  kudos_total  int default 0
  member_since date
  status       enum(active, suspended, expelled)

community.posts (id, handle_id, type enum(question,
case_discussion), title, body, status, created_at)
community.comments (id, post_id, handle_id, parent_comment_id,
body, status, created_at)
community.tags / community.post_tags        -- FD-4 hybrid
taxonomy
community.kudos
  id, target_type(post|comment), target_id, given_by_handle_id,
created_at
  unique(target_type, target_id, given_by_handle_id)   -- one
```

```
kudos per handle per item

community.reports
  id, reporter_handle_id, target_type, target_id,
  category enum(..., identifiable_patient_information, ...),
  priority boolean generated from category,
  status enum(open, actioned, dismissed), created_at

community.moderation_actions           -- immutable: INSERT-only
grant
  id, report_id nullable, target_handle_id,
  action_type enum(remove_content, warn, suspend, expel),
  moderator_id, reason, created_at

community.notifications              (handle_id, type, payload,
read_at, created_at)
community.notification_preferences   (handle_id, type,
in_app_enabled, email_enabled)
community.member_interests           (handle_id, tag, weight,
updated_at)   -- see Section 8
```

4.3 The deliberate exception: case attestations

Case-discussion attestations (PRD Section 10.3) must be "linked to the member's verified identity," not just the handle — this is the legal record that lets a confirmed patient-privacy breach be escalated to the individual's professional regulator. So `case_attestations` lives in the `identity` schema, cross-referencing `community.posts`, even though the *action* that creates it (publishing a case post) happens in the community-facing flow. It's the one place identity and community data are deliberately joined, and it's protected the same way as everything else in the `identity` schema.

4.4 Audit logging scope

`identity_access_log` captures only the three moderator-initiated reasons in PRD Section 9.4 (investigating a report, a lawful legal request, a safety escalation). Routine automated system access to the `identity` schema — sending a notification email, processing a billing charge — is normal application activity with ordinary observability, not a logged "identity access" event; conflating the two would make the audit log noisy and undermine its purpose as a trust artifact.

`identity.verification_decisions` and `community.moderation_actions` are separately immutable (per the requirement that "audit logs are immutable for verification decisions, moderation actions, and all moderator access to real identity") — enforced the same way as `identity_access_log`: an INSERT-only database grant, with `UPDATE`/`DELETE` privileges revoked at the role level.

---

5. Verification pipeline, authentication, and API design

5.1 Verification pipeline (background worker, `IdentityService`)

```
1. Registration
   POST /v1/auth/register { legal_name, email, professional_body,
registration_number, country }
   -> identity.members row created, status = pending
   -> verification job enqueued

2. Automated checks (async, worker)
   a. Register lookup - query the relevant professional register
(e.g. the Health and
      Care Professions Council (HCPC) or General Medical Council
(GMC) public register)
      by registration_number, fuzzy-match legal_name.
      Result -> identity.verification_evidence (evidence_type =
register_lookup)
   b. Identity document check via Onfido - document + facial-
similarity check, binds
      the real applicant to the claimed registration (the register
lookup alone only
      proves the registration number is real, not who is holding
it).
      Result arrives via an Onfido webhook ->
identity.verification_evidence
      (evidence_type = onfido_check)

3. Decision
   register lookup = pass AND identity check = clear  -> auto-
approve
   either fails / ambiguous / register unavailable     -> routed
to the admin review
      queue; status stays `pending` (or moves to `needs_more_info`
if the admin
      requests further evidence)
   Every transition writes an immutable row to
identity.verification_decisions.
```

Manual review (the fallback path) is worked from the same admin panel described in Section 7; both automated and manual decisions write to the same `verification_decisions` audit trail.

5.2 Authentication — passwordless by default

Rather than storing password hashes (extra attack surface, and a password-reset flow is non-trivial to build securely for a small team), the first factor is an emailed single-use magic link:

```
POST /v1/auth/request-link { email }   -> IdentityService looks up
the member,
```

```
                                        emails a single-use
signed token
GET  /v1/auth/verify?token=...          -> validates the token,
mints a session
```

Critical detail for the pseudonymity guarantee: the JSON Web Token (JWT) issued after login is scoped to the caller's **handle_id, not member_id**. Every subsequent community-facing API call (posting, commenting, awarding kudos) authenticates as a handle. The token payload never carries the real-identity linkage into browser or mobile storage — `member_id` is only ever resolved server-side, inside `IdentityService`, for the narrow set of operations that legitimately need it.

Session mechanics: a short-lived access token (around 15 minutes) plus a rotating refresh token. The web app stores the refresh token in an HttpOnly secure cookie (mitigates cross-site scripting attacks); a future native mobile client would receive it in the response body to store in platform-native secure storage instead — the same endpoints serve both, so no backend change is needed when mobile ships.

Members who are not yet `approved_verified` receive a token scoped only to a holding-status endpoint — matching the PRD's requirement that "all other statuses see a holding page only."

5.3 API design principles

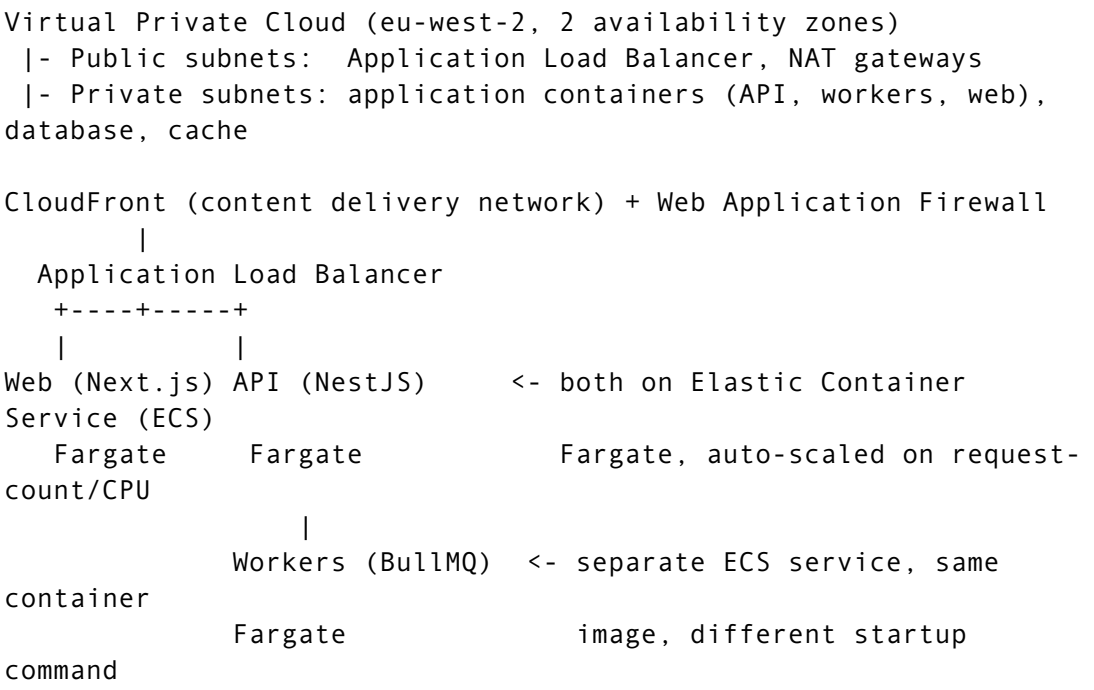
- Representational State Transfer (REST) over JSON, versioned under `/v1`, cursor-based pagination on feeds (forum listings, notifications, research feed).
- No endpoint outside `IdentityService`'s own internal calls ever returns `member_id` or `legal_name` — response shapes (Data Transfer Objects, or DTOs) for standard endpoints are built purely from `community`-schema data.
- Admin/moderation endpoints require a moderator-role claim on the JWT; any call that resolves a handle to a real identity requires a `reason_code` parameter and is exactly what populates `identity_access_log`.
- Rate limiting (Redis-backed, per-IP and per-account) on authentication and reporting endpoints.
- No server-rendered-only logic — every user-facing action is reachable via a versioned API endpoint, which is what makes native mobile clients additive later rather than a rewrite.

---

6. Infrastructure and hosting

AWS account structure: two Amazon Web Services (AWS) accounts under AWS Organizations — `production` and `staging` — genuinely separate (separate database instances, separate Onfido/payment-processor keys) so nothing in staging can touch real member data.

Region: `eu-west-2` (London) for all data stores (database, cache, file storage, compute), satisfying the UK/EU data-residency requirement. One documented exception: AWS's email-sending service (Simple Email Service, or SES) has no London endpoint — it's available in `eu-west-1` (Ireland), which is still within the EU and consistent with the residency decision, but is called out here rather than left implicit.



- **PostgreSQL** (RDS, Multi-AZ), automated backups with point-in-time recovery (30-day retention). A single instance for the MVP, but the data-access layer separates read and write database connections from day one — pointed at the same instance for now, so adding a **read replica** later (forum browsing and search are read-heavy, and the platform is being designed for faster growth) is a configuration change, not a rework.
- **Search (EPIC-C)**: start with PostgreSQL's built-in full-text search (a `tsvector` column with a Generalized Inverted Index, or GIN index) rather than standing up a dedicated search service — sufficient at the target scale and avoids operating an extra piece of infrastructure. There is a documented upgrade path to a dedicated search service (e.g. OpenSearch) if relevance or volume outgrows it, but it is not built into the MVP.
- **Secrets** (database credentials, Onfido/payment-processor keys, JWT signing keys) live in AWS Secrets Manager, injected into containers at runtime — never in environment files or source control.
- **Infrastructure as code**: AWS Cloud Development Kit (CDK) in TypeScript — the same language as the application, keeping infrastructure changes approachable for the small team and for AI-assisted development, and avoiding a second language (e.g. Terraform) to maintain.
- **Continuous integration / continuous deployment (CI/CD)**: GitHub Actions builds and tests the code, pushes a container image to Elastic Container Registry (ECR), then deploys via CDK — automatically to staging on merge to the main branch, to production behind a manual approval gate.
- **Observability**: CloudWatch for infrastructure metrics and logs; Sentry (with its European Union data-residency region explicitly selected) for application error tracking, since CloudWatch alone is weak for application-level exceptions and stack traces.

---

7. Payments, moderation tooling, and notifications

7.1 Payments — provider-agnostic by design

FD-2 (payment processor choice) is genuinely open, so `BillingService` defines a `PaymentProvider` interface rather than calling a specific processor directly from business logic:

```
interface PaymentProvider {
  createCustomer(member): ProviderCustomerId
  startSubscription(customerId, plan, trialDays):
ProviderSubscriptionId
  cancelSubscription(subscriptionId): void
  handleWebhook(rawPayload, signature): NormalizedBillingEvent
}
```

One concrete implementation is still needed to build the MVP against: `StripeProvider` is built first (strong developer experience, built-in trial/dunning handling), with `WorldPayProvider` available as a pluggable alternative if there is a commercial reason to switch. Swapping providers is then a new class and a configuration change, not a rearchitecture.

Billing data lives in its own `billing` schema (not `identity`) — `subscriptions`, `invoices`, `webhook_events` (for idempotent, replay-safe webhook processing) — keyed by `member_id`. `BillingService` has the same narrow, audited access pattern as `IdentityService`, since invoicing legitimately needs the member's real name and email, but this is kept out of the `identity` schema itself to avoid conflating "who is this person" with "what do they owe."

7.2 Moderation tooling — same web app, role-gated

Rather than a separate admin application (which would double the frontend work for a small team), the moderation/admin panel is a set of `/admin/*` routes in the same Next.js web app, gated by a moderator-role claim on the JWT, calling the same API's `/v1/admin/*` endpoints. This is a logical isolation via authentication and authorisation, not a physical one — cheap to split into a separate deployment later if that stronger isolation is ever needed.

- **Queue ordering**: reports categorised `identifiable_patient_information` are surfaced first (PRD Section 10.4's priority queue), then ordered by report age.
- **Actions** (`remove_content`, `warn`, `suspend`, `expel`) write to `community.moderation_actions` (immutable) and update `community.handles.status`. Suspension or expulsion takes effect within one access-token lifetime (about 15 minutes), since refresh tokens are revocable server-side — a suspended handle's next token refresh fails and routes them to the holding page.
- **Viewing a member's real identity** from the admin panel is a distinct, explicit action — not implicit in viewing a report — and is exactly what triggers the `reason_code`-gated `IdentityService` call and the `identity_access_log` write.

7.3 Notifications

A worker reacts to domain events (a new comment, kudos received) by writing an in-app `community.notifications` row and, if the member's preferences allow it, enqueuing an email via SES (`eu-west-1`) through `NotificationService`. This routine, automated access to a member's email address is ordinary logged application activity, not a moderation `identity_access_log` entry (Section 4.4).

Notification types for MVP: `reply`, `mention`, `kudos_received`, `verification_status_change`, and the Should-have `weekly_digest`.

---

8. Research/news feed (EPIC-I)

Re-architected onto the chosen Node.js/TypeScript stack from the design explored in `docs/AHP_Research_Feed_Design_Conversation.md` and validated by the working prototype in `prototypes/research-feed/`. The original conversation recommended ASP.NET Core; nothing in the design actually required that specific framework, and running two runtimes for a small team adds real operational overhead without a compelling reason — so this reuses the same NestJS modules, PostgreSQL database, Redis queue, and BullMQ workers already established above.

```
research.articles
  id, doi, pmid, other_ids jsonb, title, abstract, journal,
  published_date, evidence_type, open_access boolean,
  source_refs jsonb, created_at

research.ingestion_cursors
  source_name, last_run_at, last_cursor        -- per-source
incremental fetch

community.member_interests
  handle_id, tag, weight, updated_at            -- weighted
interests, not
                                                  -- simple
keywords
```

- **Source adapters** — one NestJS provider per external literature source (Europe PMC, OpenAlex, Semantic Scholar, Crossref/PubMed), each implementing a common `fetchSince(cursor): RawArticle[]` interface. Same pattern as `PaymentProvider` — a new source is a new adapter, not a core change.
- **Ingestion** — BullMQ repeatable jobs per adapter (every few hours), using the same worker infrastructure as verification and notifications.
- **Deduplication** — priority order Digital Object Identifier (DOI), then PubMed Identifier (PMID), then other identifiers, then normalised title/year, then fuzzy matching — applied at upsert time into `research.articles`, matching the original design conversation's approach.
- **Classification (MVP)** — rule-based proximity-window keyword matching against a controlled taxonomy, exactly as the working prototype already validates. No vector embeddings for MVP: PostgreSQL's `vector` extension (pgvector) is a documented, cheap upgrade path if rule-based classification proves too noisy at scale, but building it now would be

speculative given the prototype already demonstrates the simpler approach works.

- **Scoring** — topic match plus evidence-type weighting (systematic review > randomised controlled trial > cohort study > case report) plus recency, computed once at ingestion time and stored, not recalculated on every request — keeps the feed endpoint fast as the article corpus grows.
- **Feed API** — `GET /v1/research-feed` combines each member's `member_interests` weights with precomputed article scores at query time, and returns the same style of explanation string as the prototype ("Recommended because it matches your interest in X…").
- **Data boundary** — `member_interests` is handle-scoped (a clinical-interest profile, not real identity data), so it lives in the `community` schema under the same access rules as everything else there; no `identity`-schema involvement.
- **Quality/safety filtering** — `docs/research-feed-sources-and-roadmap.md` identifies concrete, free sources to close this gap rather than leaving it unscoped: the Directory of Open Access Journals (DOAJ) for a predatory-journal legitimacy check, and the Retraction Watch database (now folded into Crossref's open API) for flagging withdrawn papers. Neither is integrated yet — tracked as an EPIC-I pre-launch item — but the sourcing question is answered, only the integration work remains.
- **PEDro (Physiotherapy Evidence Database)** is flagged in the same roadmap doc as the best domain-specific fit for this audience (68,000+ physiotherapy trials/reviews, each quality-rated on the PEDro scale) but with no confirmed public API — worth a direct enquiry to the PEDro team (University of Sydney/NeuRA) before committing engineering time to it.
- **Carried-over gap**: no licensing review has been done for redistributing article abstracts commercially (relevant if a commercial source like Elsevier/Scopus/Cochrane is ever added) — still open.

---

9. Non-functional requirements

Security - Transport Layer Security (TLS) everywhere; encryption at rest on the database, cache, and file storage (S3 server-side encryption). - JWT signing keys held in Secrets Manager with rotation; refresh tokens revocable server-side. - AWS WAF on CloudFront (common attack patterns, rate limiting) plus application-level rate limiting on authentication and reporting endpoints. - Automated dependency vulnerability scanning in CI. - A specific regression test class: automated tests asserting that the `community`-schema database role genuinely cannot query the `identity` schema — the core trust guarantee should be enforced by a test that fails the build, not by code review alone.

Privacy and compliance - No PHI, enforced by policy and by the case-discussion attestation/checklist gate (Section 4.3). - Right-to-erasure default (worth explicit legal review before launch, as this is a policy decision as much as a technical one): on a deletion request, the `identity` record is hard-deleted; the handle's community content is retained but the handle is marked `deleted_member` with no possibility of re-linking to an identity. This preserves the forum's value as a knowledge archive while honouring the erasure request for real PII specifically. - A Data Protection Impact Assessment (DPIA) is worth commissioning before launch, given the verification pipeline processes professional-registration data at volume.

Availability and performance - Target roughly 99.5% uptime for the MVP — deliberately not over-promising a level a small team cannot realistically staff for. - Multi-AZ database, application containers spread across two

availability zones, rolling deployments with health checks. - 95th-percentile API latency target under 300 milliseconds for standard reads; feed and search queries backed by appropriate indexes.

Observability - Structured JSON logging with a correlation ID threaded from an API request through to any worker jobs it triggers, so a single user action is traceable end-to-end. - CloudWatch alarms on error rate, queue depth, and database connection saturation; Sentry for application exception tracking.

Disaster recovery - Database point-in-time recovery; Recovery Point Objective (RPO) of around 5 minutes, Recovery Time Objective (RTO) of a few hours — pragmatic for MVP scale, not a “five nines” commitment.

Testing strategy - Unit tests per module; integration tests against a dockerised PostgreSQL instance in CI; contract tests for the pluggable interfaces (`PaymentProvider`, source adapters) so swapping an implementation cannot silently break the contract.

---

10. Epic-to-module map

Each row below is expected to become its own per-epic technical spec, building directly on the foundations in this document.

Epic	NestJS module(s)	Primary schema(s)
EPIC-A — Verification	`identity` (IdentityService)	`identity`
EPIC-B — Handles/profile	`handles`	`community`
EPIC-C — Forum	`forum`	`community`
EPIC-D — Kudos	`kudos` (part of `forum` or standalone)	`community`
EPIC-E — Case discussions	`case-discussions`	`community` + `identity.case_attestations`
EPIC-F — Moderation	`moderation` + admin panel routes	`community`, gated `identity` access
EPIC-G — Notifications	`notifications` (NotificationService)	`community`, gated `identity` access (email)
EPIC-H — Subscription/ payments	`billing` (BillingService)	`billing`, gated `identity` access
EPIC-I — Research feed	`research-feed`	`research`, `community.member_interests`

---

11. Open follow-ups

Items surfaced during this design that need action outside this document:

- **PRD update**: EPIC-I (research/news feed) should be added to the PRD’s MVP epic list (Section 6.1) — it was not part of the original

MoSCoW scope and this is a real scope addition, worth Paul Gouge and Andrew Renshaw's awareness.

- **Legal review**: the right-to-erasure default described in Section 9 (hard-delete identity, retain de-linked community content) should be confirmed with legal counsel before launch.
- **DPIA**: commission a Data Protection Impact Assessment before launch given the volume of professional-registration data processed.
- **Research feed gaps**: DOAJ and Retraction Watch integration (sources identified, not yet built) and abstract-redistribution licensing review (source unidentified) remain open before EPIC-I can ship (Section 8). A direct enquiry to PEDro about API access is also worth making given it's the best domain-specific fit found so far.
- **FD-2 (payment processor)**: this document designs around either Stripe or WorldPay via the `PaymentProvider` interface (Section 7.1), but a concrete choice is still needed to build the first implementation against.